FORMATION

Analyse de données avec Python

Jour 3, matin
Organisation de projet, code robuste et ligne de commande

Cours et TP5 en alternance : du notebook du TP3 à la commande ventes

Le déroulé de la matinée

1. **uv** — puis TP5, étape 0
2. **Structure**
3. **Modules** — puis TP5, étape 1
4. **Types**
5. **Exceptions** — puis TP5, étape 2

Pause

6. **Tests** — puis TP5, étapes 3 et 4
7. **argparse**
8. **logging**
9. **Config** — puis TP5, étapes 5 à 7

9h00
début

10h30
pause

12h30
fin

Objectif : faire du TP3 un outil autonome, en alternant blocs de cours et étapes du TP5.

01

De pip et venv à uv

Pourquoi | uv init | uv add et uv run | uv sync

1 Pourquoi uv

1

Un seul outil

Environnement, installation, verrouillage des versions et même installation de Python.

2

Très rapide

L'installation des bibliothèques prend quelques secondes là où pip prend des minutes.

3

Reproductible

`pyproject.toml` décrit le projet, `uv.lock` fige les versions exactes pour toutes les machines.

Installer uv, puis vérifier avec `uv --version` :

```
# Ubuntu, macOS
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env

# Windows PowerShell
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

1 Créer un projet

Terminal

```
uv init --package ventes
cd ventes
```

Fichiers créés

```
ventes/
  .git/                dépôt git
  .gitignore
  .python-version      3.12 sur les postes
  README.md
  pyproject.toml
  src/
    ventes/
      __init__.py
```

pyproject.toml

```
[project]
name = "ventes"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = []

[project.scripts]
ventes = "ventes:main"
```

`--package` crée un projet installable, avec le code dans `src/`, importable depuis les notebooks et exécutable en commande.

uv initialise aussi un dépôt git et note dans `.python-version` la version de Python utilisée.

1 Ajouter et exécuter

```
uv add pandas pyarrow pyyaml

uv run python mon_script.py
uv run ventes
```

`uv add` crée le `.venv`, installe, inscrit dans `pyproject.toml` et met à jour `uv.lock`
`uv run` exécute dans l'environnement du projet, **sans activation**

pip et venv	uv
`python -m venv` puis activation	automatique
`pip install pandas`	`uv add pandas`
`pip freeze > requirements.txt`	automatique
`python script.py`	`uv run python script.py`

Pour récupérer un projet existant, comme le squelette du TP5 : `uv sync` crée l'environnement et installe tout ce que déclare `pyproject.toml`.

1 Pratique : TP5, étape 0

Préparer l'environnement du projet, dans exercices/tp05-script-cli

Terminal

```
uv --version
uv sync
uv run pytest
```

`uv sync` crée `.venv` avec le Python 3.12 du poste et installe les dépendances du `pyproject.toml`
Aucune activation : `uv run` suffit

Résultat attendu de uv run pytest

```
5 failed, 3 errors in 9.17s
```

C'est normal : les fonctions contiennent encore `raise NotImplementedError`. À la fin du TP, tous les tests passent.

02

Structurer un projet data

data | notebooks | src | tests | outputs

2 Une structure type

```
ventes/
  pyproject.toml
  uv.lock
  README.md
  config.yaml
  data/
    brut/
  notebooks/
    01_exploration.ipynb
  src/
    ventes/
      __init__.py
      chargement.py
      indicateurs.py
      erreurs.py
      config.py
      cli.py
  tests/
  outputs/
```

Dossier	Règle
`data/brut`	jamais modifié : tout se recalcule à partir de lui
`notebooks`	numérotés, ils importent le code de `src`
`src`	le code réutilisable, sans chemin en dur
`tests`	un fichier de test par module
`outputs`	régénérable : on peut tout supprimer et relancer

Le dossier `src/` oblige à installer le paquet : il s'importe alors de la même façon dans un notebook, un test ou un script.

03

Du notebook au module

Notebook ou script | Extraire des fonctions | Importer

3 Notebook ou module ?

	Notebook	Module et script
Idéal pour	explorer, visualiser, raconter	un traitement répété, automatisé
Exécution	cellule par cellule, ordre libre	de haut en bas, toujours pareil
Réutilisation	copier-coller	`import`
Tests	difficiles	naturels
Versionnement	diff peu lisible	diff clair

Les deux se complètent : le code stable part dans des modules, le notebook garde l'exploration et la restitution.

3 Extraire une fonction

Dans le notebook du TP3

```
fichiers = sorted(Path("ressources/ventes")
                  .glob("*.csv"))
ventes = pd.concat(
    [pd.read_csv(f, parse_dates=["date"])
     for f in fichiers],
    ignore_index=True,
)
```

Dans src/ventes/chargement.py

```
def charger_ventes(dossier: Path,
                   pattern: str = "*.csv"
                   ) -> pd.DataFrame:
    fichiers = lister_fichiers(dossier, pattern)
    ventes = pd.concat(
        [charger_fichier_ventes(f) for f in fichiers],
        ignore_index=True,
    )
    verifier_unicite(ventes, "id_commande", "ventes")
    return ventes
```

Une bonne fonction fait une seule chose, reçoit ses données en paramètres et renvoie un résultat, sans rien afficher ni lire de variable extérieure.

3 Modules, paquets et imports

Un **module** : un fichier `.py`
Un **paquet** : un dossier de modules avec un `__init__.py`
`if __name__ == "__main__":` : code exécuté seulement quand le module est lancé directement

```
from ventes.chargement import charger_ventes
from ventes.erreurs import ColonnesManquantesError
```

Dans un notebook du projet

```
uv add --dev jupyterlab
uv run jupyter lab
```

```
%load_ext autoreload
%autoreload 2

from ventes.chargement import charger_ventes
```

`autoreload` recharge les modules modifiés sans redémarrer le noyau.

3 Pratique : TP5, étape 1

Repérer dans le notebook du TP3 les blocs à extraire

Copier le notebook du TP3 dans `notebooks/`

1. L'ouvrir dans l'environnement du projet

Pour chaque fonction de `chargement.py` et `indicateurs.py`, noter en commentaire les cellules qu'elle remplace

Terminal

```
uv add --dev jupyterlab
uv run jupyter lab
```

Résultat attendu : chaque fonction déclarée dans `src/ventes` a sa correspondance dans le notebook.

04

Les annotations de type

Syntaxe | Usage | mypy

4 Annoter une fonction

```
def taux_realisation(ca: float,
                     objectif: float) -> float:
    return ca / objectif


def lister_fichiers(dossier: Path,
                    pattern: str = "*.csv"
                    ) -> list[Path]:
    ...
```

Annotation	Signifie
`list[str]`	liste de textes
`dict[str, float]`	dictionnaire
`str | None`	texte ou rien
`Path`	chemin
`pd.DataFrame`	tableau pandas
`-> None`	ne renvoie rien

La signature dit ce que la fonction attend et ce qu'elle renvoie, sans lire son code. L'éditeur s'en sert pour compléter et signaler les erreurs.

4 mypy vérifie avant l'exécution

```
lister_fichiers("ressources")
# Python accepte l'appel : l'erreur
# arrive plus loin, sur .glob
```

Terminal

```
uv add --dev mypy pandas-stubs
uv run mypy src
```

Résultat

```
exemple.py:14: error: Argument 1 to
"lister" has incompatible type "str";
expected "Path"  [arg-type]
Found 1 error in 1 file
```

Python ne vérifie pas les annotations à l'exécution. mypy trouve l'incohérence sans lancer le code.

05

Les exceptions

try et except | raise | Exceptions du projet

5 Gérer une exception

```
def convertir_prix(texte: str) -> float | None:
    try:
        valeur = float(texte)
    except ValueError:
        print(f"Valeur non numérique : {texte!r}")
        return None
    else:
        # seulement si aucune exception
        return round(valeur, 2)
    finally:
        # dans tous les cas
        pass

convertir_prix("12,5")   # message, None
```

Exception	Cas typique
`ValueError`	`float("abc")`
`TypeError`	`"3" + 2`
`KeyError`	clé ou colonne absente
`FileNotFoundError`	fichier introuvable
`ZeroDivisionError`	division par zéro

5 Bonnes pratiques et raise

Attraper

Une exception **précise** : `except ValueError`
Un bloc `try` **court**, autour de la seule instruction risquée

- Jamais d'erreur ignorée en silence

À éviter : `except:` sans type, qui attrape aussi les fautes de frappe.

Lever

```
def taux_remise(pourcentage: float) -> float:
    if not 0 <= pourcentage <= 100:
        raise ValueError(
            f"Pourcentage invalide : {pourcentage}"
        )
    return pourcentage / 100
```

Mieux vaut échouer tôt avec un message clair que produire en silence un résultat faux.

5 Des exceptions propres au projet

```
class VentesError(Exception):
    """Erreur de base du traitement des ventes."""


class ColonnesManquantesError(VentesError):
    def __init__(self, fichier: Path,
                 manquantes: set[str]) -> None:
        self.fichier = fichier
        self.manquantes = manquantes
        super().__init__(
            f"{fichier.name} : colonnes manquantes "
            f"{sorted(manquantes)}"
        )
```

Des **messages métier** plutôt qu'une `KeyError` trois fonctions plus loin

`except VentesError` attrape toutes les erreurs du projet, et seulement elles

- L'exception transporte le fichier et les colonnes en cause

```
raise ConfigurationError(...) from erreur
```

`from` garde la trace de l'erreur technique d'origine.

5 Où attraper, et quand avertir

1

Les fonctions lèvent

Elles signalent le problème et ne l'attrapent pas si elles ne savent pas quoi en faire.

2

Le point d'entrée attrape

La commande attrape toutes les `VentesError`, affiche un message clair et s'arrête avec un code d'erreur.

3

Avertir si on peut continuer

Colonne manquante : exception.
Produits hors catalogue : avertissement dans le journal, et on continue.

5 Pratique : TP5, étape 2

Les exceptions du projet, dans src/ventes/erreurs.py

Compléter `DossierVideError`, écrire `ColonnesManquantesError`et `CleNonUniqueError`, qui héritent de `VentesError`

- Un message qui permet de comprendre le problème sans lire le code

Résultat attendu, dans uv run python

```
>>> from ventes.erreurs import ColonnesManquantesError, DossierVideError
>>> print(DossierVideError(Path("data/brut/ventes"), "*.csv"))
aucun fichier *.csv dans data/brut/ventes
>>> erreur = ColonnesManquantesError(
...     Path("data/brut/ventes/ventes_2025_10.csv"), {"quantite"})
>>> print(erreur)
ventes_2025_10.csv : colonnes manquantes ['quantite']
>>> erreur.manquantes
{'quantite'}
```

06

Tester et vérifier le code

pytest | Tests de recette | ruff

6 Un test avec pytest

```
def test_enrichir_garde_toutes_les_ventes(tables):
    ventes, produits, regions = tables
    df = enrichir(ventes, produits, regions)
    assert len(df) == len(ventes)
    assert df.loc[df["id_produit"] == "P99",
                  "categorie"].item() == "Non référencé"
```

Terminal

```
uv add --dev pytest
uv run pytest
```

Résultat

```
collected 7 items

tests/test_chargement.py ....    [ 57%]
tests/test_indicateurs.py ...    [100%]

========== 7 passed in 0.17s ==========
```

Un fichier `tests/test_*.py`, des fonctions `test_...`

- De petites données construites à la main, au résultat connu
- Le cas normal, les cas limites, les erreurs attendues

6 Fixtures, fichiers temporaires et erreurs

Une fixture partagée

```
@pytest.fixture
def tables():
    ventes = pd.DataFrame({...})
    produits = pd.DataFrame({...})
    regions = pd.DataFrame({...})
    return ventes, produits, regions
```

tmp_path et pytest.raises

```
def test_lister_fichiers_dossier_vide(
        tmp_path: Path) -> None:
    with pytest.raises(DossierVideError):
        lister_fichiers(tmp_path)
```

`tmp_path` : un dossier vide, créé pour le test et nettoyé ensuite.
`pytest.raises` vérifie qu'une exception est levée.

6 Inspecter une erreur, comparer des décimaux

`pytest.raises ... as erreur`

```
with pytest.raises(ColonnesManquantesError) as erreur:
    charger_fichier_ventes(fichier)
assert erreur.value.manquantes == {"quantite"}
```

`pytest.approx` pour les décimaux

```
assert taux_realisation(70, 100) == pytest.approx(0.7)
total = ventes["montant"].sum()
assert total == pytest.approx(409501.59, abs=0.01)
```

Lancer une partie des tests

```
uv run pytest tests/test_chargement.py
uv run pytest -k unicite
uv run pytest -x
```

- un seul fichier

`-k` : les tests dont le nom contient « unicite »
`-x` : s'arrêter au premier échec

Pas de == entre décimaux, à cause des arrondis : `pytest.approx` tolère un petit écart.

6 Les tests de recette

Ils vérifient les résultats attendus sur les **vraies données**

- Lignes chargées, total du chiffre d'affaires, lignes produites
- Plus lents, mais ils garantissent les mêmes chiffres que le notebook d'origine

Extrait de tests/test_recette.py

```
def test_les_9_fichiers_donnent_2182_ventes() -> None:
    ventes = charger_ventes(DONNEES / "brut" / "ventes")
    assert ventes.shape == (2182, 8)
    assert ventes["fichier"].nunique() == 9
```

Le TP5 en fournit un fichier, `tests/test_recette.py`, à ne pas modifier : ses tests échouent au départ et doivent tous passer à la fin.

6 ruff : lint et formatage

```
uv add --dev ruff

uv run ruff check .         # signale
uv run ruff check --fix .   # corrige
uv run ruff format .        # reformate
```

Résultat

```
F401 [*] `os` imported but unused
 --> exemple.py:1:8
  |
1 | import os
  |        ^^
```

Imports inutiles, variables non utilisées, comparaisons douteuses, style incohérent : ruff les détecte en une fraction de seconde.

6 Pratique : TP5, étapes 3 et 4

Étape 3 : le chargement

Les quatre fonctions de `chargement.py`
Les exceptions de l'étape 2, un journal avec `logger.info`
Compléter `tests/test_chargement.py`

```
uv run pytest tests/test_chargement.py
uv run pytest tests/test_recette.py -k 2182
```

Étape 4 : les indicateurs

`enrichir`, `indicateurs_mois_region`, `filtrer_periode`
`logger.warning` pour les produits hors catalogue
Compléter `tests/test_indicateurs.py`

```
uv run pytest
```

Résultat attendu : tous les tests passent, sauf les deux tests de recette qui appellent la commande, pas encore écrite.

07

Une commande avec argparse

Paramètres | Aide automatique | Code de sortie

7 Déclarer les paramètres

```
parser = argparse.ArgumentParser(
    prog="ventes",
    description="Consolide les ventes mensuelles.",
)
parser.add_argument("-c", "--config", type=Path,
                    default=Path("config.yaml"))
parser.add_argument("-o", "--sortie", type=Path)
parser.add_argument("--debut", help="AAAA-MM")
parser.add_argument("--fin", help="AAAA-MM")
parser.add_argument("-v", "--verbose",
                    action="store_true")

args = parser.parse_args()
args.config, args.verbose
```

Option	Rôle
`"dossier"`	positionnel, obligatoire
`"--config"`	option, facultative
`type=Path`	conversion
`default=`	valeur par défaut
`action="store_true"`	interrupteur
`choices=[...]`	valeurs permises

7 Déclarer la commande et son code de sortie

pyproject.toml

```
[project.scripts]
ventes = "ventes.cli:main"
```

Terminal

```
uv run ventes --help
uv run ventes --debut 2025-07 --fin 2025-08
```

```
def main(argv: list[str] | None = None) -> int:
    args = creer_parser().parse_args(argv)
    configurer_console(args.verbose)
    try:
        executer(args)
    except VentesError as erreur:
        logger.error("%s", erreur)
        return 1
    return 0
```

0 : succès. 1 : échec. C'est ce code que lisent les tâches planifiées et les pipelines.

08

Journaliser avec logging

Niveaux | Dans les modules | Dans le point d'entrée

8 Pourquoi pas print

Un **niveau** de gravité, pour filtrer
L'**heure** et l'**origine** de chaque message
La console **et** un **fichier** qui garde l'historique

Niveau	Usage
`DEBUG`	détail pour comprendre un problème
`INFO`	étapes normales
`WARNING`	anomalie, le traitement continue
`ERROR`	le traitement a échoué
`CRITICAL`	le programme ne peut plus fonctionner

Dans chaque module

```
logger = logging.getLogger(__name__)

logger.info("%d ventes chargées",
len(ventes))
```

8 Configurer une seule fois, au point d'entrée

```
FORMAT_JOURNAL = "%(asctime)s %(levelname)-8s %(name)s : %(message)s"

logging.basicConfig(level=logging.DEBUG if verbose else logging.INFO, format=FORMAT_JOURNAL)

journal = logging.FileHandler("outputs/ventes.log", encoding="utf-8")
journal.setFormatter(logging.Formatter(FORMAT_JOURNAL))
logging.getLogger().addHandler(journal)
```

Résultat

```
2026-09-26 14:56:50 INFO     ventes.chargement : 9 fichiers trouvés dans data/brut/ventes
2026-09-26 14:56:50 INFO     ventes.chargement : 2182 ventes chargées
2026-09-26 14:56:50 WARNING  ventes.indicateurs : 17 ventes de produits hors catalogue : P17
2026-09-26 14:56:52 INFO     ventes : 54 lignes d'indicateurs écrites dans outputs/indicateurs_mois_region.parquet
```

09

Séparer la configuration du code

YAML | .env | Priorités

9 Chaque valeur à sa place

Où	Quoi
le code	la logique, des valeurs par défaut
`config.yaml`	chemins, paramètres stables
les arguments	ce qui change à chaque exécution
`.env`	les secrets

Priorité : argument, puis configuration, puis valeur par défaut.

config.yaml

```
donnees:
  ventes: data/brut/ventes
  produits: data/brut/produits.csv
  regions: data/brut/regions.csv

sorties:
  indicateurs: outputs/indicateurs_mois_region.parquet
  journal: outputs/ventes.log
```

9 Lire la configuration et les secrets

YAML

```
import yaml

def charger_config(chemin: Path) -> dict[str, Any]:
    if not chemin.exists():
        raise ConfigurationError(
            f"introuvable : {chemin}")
    with chemin.open(encoding="utf-8") as f:
        config = yaml.safe_load(f) or {}
    # vérifier les clés obligatoires
    return config
```

.env

```
# .env
API_CLE=abc123

from dotenv import load_dotenv
load_dotenv()
cle = os.getenv("API_CLE")
```

Le `.env` ne se versionne jamais : il va dans le `.gitignore`, et on fournit un `.env.exemple` aux valeurs fictives.

9 Pratique : TP5, étapes 5 à 7

5

La configuration

`config.yaml` avec `donnees` et `sorties`, `charger_config` et ses trois tests. Attendu : `3 passed`.

6

La commande

Dans `cli.py` : les options, les journaux console et fichier, le traitement, le code de sortie.

7

De bout en bout

La commande, une période, un fichier défectueux, puis tous les tests, ruff et mypy.

```
uv run pytest tests/test_config.py
uv run ventes --help
uv run ventes
```

9 Pratique : TP5, étape 7, de bout en bout

Commande	Résultat attendu
`uv run ventes`	2182 ventes, 17 hors catalogue, 54 lignes d'indicateurs ; les mêmes lignes dans `outputs/ventes.log`
`uv run ventes --debut 2025-07 --fin 2025-08 --sortie outputs/ete.parquet`	12 lignes d'indicateurs : 2 mois et 6 régions
un fichier sans `quantite` copié en `ventes_2025_10.csv`, puis `echo $?`	`colonnes manquantes ['quantite']` au niveau ERROR, puis `1`
`uv run pytest`	tous les tests passent, recette comprise
`uv run ruff check .`	`All checks passed!`
`uv run mypy src`	`Success: no issues found`

TP5 : d'un notebook à un script en ligne de commande

Dans exercices/tp05-script-cli : étapes 0 à 7 et résultats attendus dans le README

1

Extraire

Écrire les fonctions typées déclarées dans `src/ventes`, à partir du notebook du TP3.

2

Sécuriser

Exceptions du projet, tests pytest, vérifications ruff et mypy.

3

Livrer

La commande `uv run ventes`, avec paramètres, journal et `config.yaml`, qui fonctionne de bout en bout.

Le fichier fourni `tests/test_recette.py` vérifie les chiffres sur les vraies données : 2182 ventes, 54 lignes d'indicateurs, un code de sortie à 1 avec un fichier défectueux. Le TP est terminé quand tous les tests passent.

À retenir

`uv init --package`, `uv add`, `uv run`
`data/brut` intouchable, `src`, `tests`, `outputs` régénérable

- Le code stable en fonctions, puis en modules importés
- Les annotations documentent, mypy vérifie
- Exceptions précises, levées tôt, attrapées au point d'entrée

Un test par comportement, `uv run pytest`, `pytest.approx` pour les décimaux
`ruff check` et `ruff format` pour un code homogène
argparse, `[project.scripts]` et un code de sortie
`getLogger(__name__)` dans les modules, configuration au point d'entrée
Chemins en configuration, secrets dans `.env`